

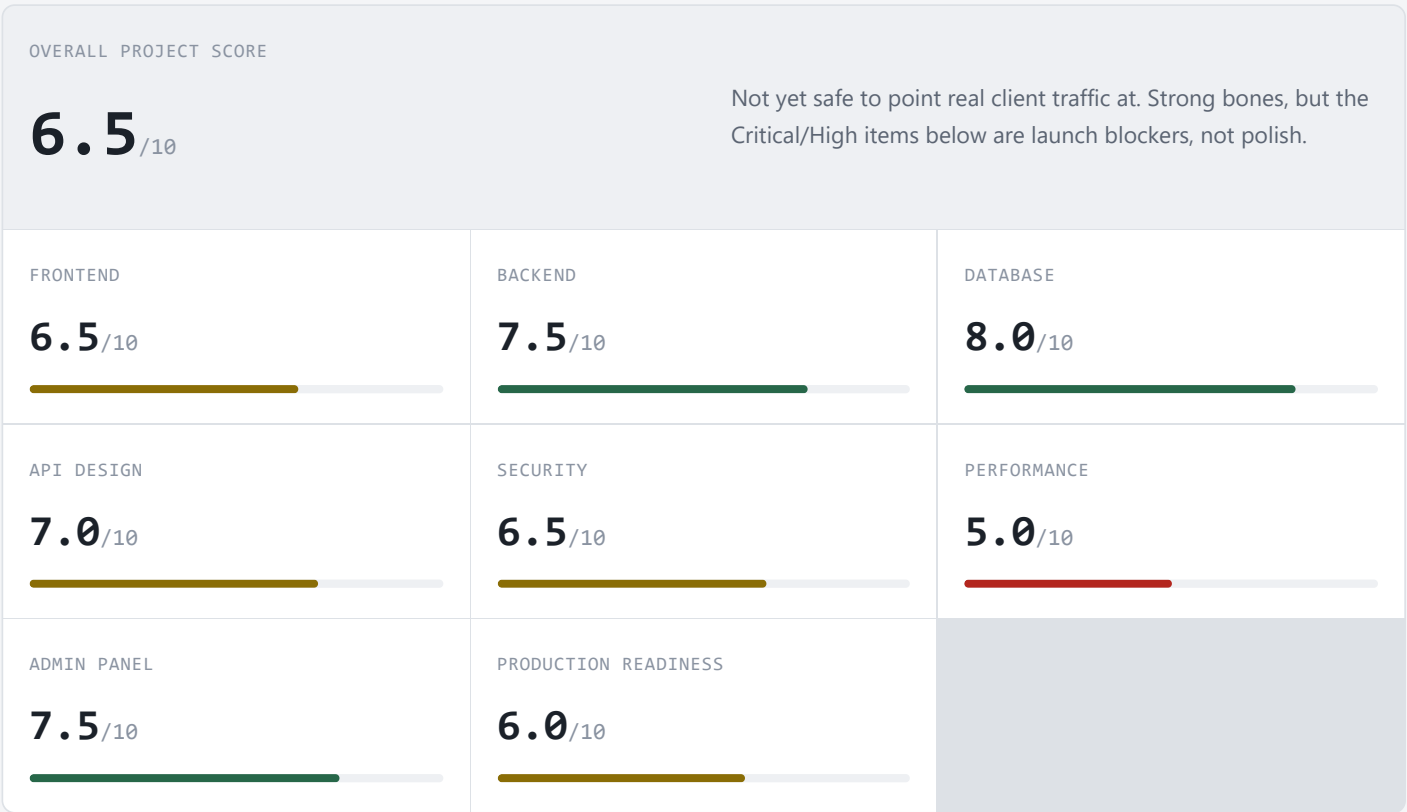
Paawani & Associates — Site & Admin Panel

A code-level review of the public website, admin panel, REST API, MySQL schema, and Calendly integration ahead of serving real clients — architecture, security, performance, and functional completeness.

Stack: React/Tailwind · Flask · MySQL · Calendly **Date:** 1 Aug 2026 **Files reviewed:** 103 backend · 328 frontend

00 Executive Summary

The engineering foundations are genuinely solid — the auth system, database schema, and admin CRUD scaffolding are better than most pre-launch projects this size. The gaps that remain are concrete and fixable: an unauthenticated path into the appointment system, three oversized images that will make the homepage unusable on mobile, no automated tests, and a handful of config defaults that need to be pinned before anything touches a production host.



Methodology note: this is a rigorous static, evidence-based code audit — every finding below cites a real file and line, and the highest-severity claims (RoleGuard usage, the 24.6MB hero image, the missing webhook route, the SVG upload allow-list) were independently re-verified by direct inspection, not taken on trust. It is **not** a live click-through QA pass against a running instance with a real MySQL database, a Calendly sandbox, and a live email provider — that requires credentials and an environment this review didn't have. Section 9 lists the manual QA checklist to run before go-live.

Public website and admin panel (React + Tailwind). Admin panel findings are cross-referenced in §7; this section leads with the public site, which carries the most launch-blocking issues.

PUBLIC WEBSITE

CRITICAL Performance

The homepage hero renders a **24.6MB PNG** eagerly, above the fold, with no lazy-loading.

FILE `frontend/src/website/pages/Home/components/HomeHero.jsx:8,147-153`

WHY Every homepage visit downloads 24MB before the largest-contentful-paint can complete. On a 4G connection this is 30+ seconds — the homepage is effectively broken for mobile visitors, and Google's Core Web Vitals ranking will penalize it accordingly.

FIX Re-export as WebP/AVIF at the actual display size (target <200KB) — the partner photos elsewhere in the repo already follow this pattern, just apply it here.

HIGH Performance

A 9.88MB PNG is used as a full-bleed background on both the About and Blog hero sections.

FILE `pages/About/components/AboutHero.jsx:22-26` , `pages/Blog/BlogListing/components/BlogHero.jsx:19-25`

WHY Two of the site's main traffic pages each pay a ~10MB tax for a decorative background — same LCP problem as the hero image, on pages that matter for SEO.

FIX One shared, optimized WebP background (a few hundred KB) reused across both heroes instead of two independent oversized PNGs.

HIGH Performance / Deploy Hygiene

The public/ folder (copied verbatim into every build) ships several oversized, in one case entirely unused, assets.

FILE `public/partners/amit.png` (33MB, unreferenced — `data/partners.js:13` actually points at `amit.webp`);
`public/article/Akshit Arora-.mp4` (59MB placeholder video, played from
`LifeAtSAA/components/VideoModal.jsx:143-154`)

WHY The video specifically streams to any visitor who clicks a "watch" card on Life@SAA — a 59MB placeholder clip has no place going live for real clients, and the unused 33MB image is pure dead weight on every deploy.

FIX Delete the unreferenced file; replace the placeholder video with a properly compressed (<10MB) clip or pull the feature until real footage exists.

CRITICAL SEO

The document title is the literal placeholder "profile-project", and the per-page <Seo> component is wired into only 4 of ~16 routes.

- FILE `frontend/index.html:8` ; absent from `Home.jsx` , `About.jsx` , `ContactPage.jsx` , `Career.jsx` , `BlogListing.jsx` , and every hand-built Service page
- WHY Confirmed directly — the homepage and most service/marketing pages show "profile-project" as the browser tab and search-result title, with no meta description, for a firm about to serve real accounting clients.
- FIX Set a real default title in `index.html` , then add `<Seo title=... description=... canonicalPath=... />` to every routed page, not just the four that have it.

HIGH Routing

No catch-all path: "*" route and no `errorElement` anywhere in the router.

- FILE `frontend/src/website/routes/AppRoutes.jsx:20-42`
- WHY A typo'd URL, stale bookmark, or old Google link falls through to React Router's bare default screen ("Unexpected Application Error!") instead of a branded 404 — and any unhandled render exception anywhere hits the same generic crash screen.
- FIX Add a branded 404 as the catch-all child route, plus an `errorElement` on the root layout route.

```
{ path: "*", element: <NotFound /> }  
// on the root/layout route:  
errorElement: <AppErrorBoundary />
```

MEDIUM Legal / Compliance

The Contact form's mandatory "I agree to the Privacy Policy" checkbox links to `href="/"` — the homepage. No privacy policy page exists anywhere in the codebase.

- FILE `pages/Contact/components/ContactForm.jsx:339`
- WHY Visitors are asked to consent to a document that doesn't exist before submitting name/phone/email to a live accounting firm — a real gap under India's DPDP Act 2023 and basic user trust, confirmed by `find ... -iname "*privacy*"` returning nothing.
- FIX Build a real `/privacy-policy` route (and a Terms page) and point the link there — see §10.

MEDIUM Performance

Zero route-level code splitting — every page (Home, About, Contact, Appointment, ~15 service pages, Career, Blog, Insights) is statically imported into one bundle.

- FILE `frontend/src/website/routes/AppRoutes.jsx:4-17`
- WHY A homepage visitor downloads the JS for the job-application form, all service pages, and blog machinery in the same first-load bundle, for no benefit.
- FIX Wrap route element imports in `React.lazy()` + `<Suspense>` , especially across the long tail of Service pages.

MEDIUM Code Quality

Form/animation boilerplate (EASE, fadeUp, inputBaseClasses, error-message markup) is copy-pasted verbatim across three separate forms and has already begun drifting.

FILE `Contact/components/ContactForm.jsx:23-61` , `Appointment/components/BookingSection.jsx:27-48` ,
`Career/components/ApplyNow.jsx:21-45`

WHY Contact/Appointment use `text-black` for errors, Career uses `text-secondary` — the copies have already diverged, so a future fix needs to be hunted down and applied three times.

FIX Extract a shared `FormField` / `ErrorMessage` module and a shared `motion/variants.js` for the animation constants.

LOW Dead Code

`GenericServicePage.jsx` is a fully built, working page that is never imported anywhere — `DynamicServicePage.jsx` handles `/services/:slug` instead.

FILE `pages/Services/GenericServicePage.jsx`

FIX Delete it, or if it's meant to replace `DynamicServicePage` , wire it in and remove the other.

LOW Accessibility

Decorative hero backgrounds are inconsistently marked up — some correctly use `alt="" aria-hidden="true"`, others give a redundant real alt with no aria-hidden for the same kind of purely-decorative image.

FILE Correct: `BlogHero.jsx:22-23` , `LifeHero.jsx:21-22` . Inconsistent: `AboutHero.jsx:24` (`alt="About Background"`),
`ContactHero.jsx:22` , every Service-page `Hero.jsx`

FIX Standardize on `alt="" aria-hidden="true"` for all purely-decorative background images.

✓ Strengths — public site

- The **Calendly embed** (`components/common/CalendlyEmbed.jsx` , `utils/calendly.js`) is the most carefully engineered part of the whole codebase — singleton script loading, a calendar-shaped skeleton, an explicit retry state if the script fails to load, and a `MutationObserver` -based popup tracker since Calendly doesn't emit a documented "closed" event.
- Every data-fetched section (stats, testimonials, openings, blog) implements loading/error/empty states with `aria-busy` / `aria-live` / `role="alert"` — nothing fails silently.
- Forms consistently disable submit during `isSubmitting` , show a spinner, and give distinct success/error messaging — no duplicate-submit risk found on the client side.
- Image `alt` discipline is otherwise genuinely good, and content images use `loading="lazy" decoding="async"` .
- `NewsletterUnsubscribe.jsx` deliberately requires a button click rather than acting on page load, specifically to avoid mail-scanner link-prefetch mass-unsubscribing users — a subtle, correct call.

ADMIN PANEL — SEE §7 FOR THE FULL WRITE-UP

The admin panel's frontend findings (missing route-level `RoleGuard`, inconsistent delete permissions, required prop not applied to inputs, no focus trap in dialogs) are detailed together with the rest of the admin panel review in §7, since they're inseparable from the CRUD/authorization discussion there.

Backend Review

Flask app factory, config, middleware, and the auth blueprint. Domain-logic blueprints (contact, career, blog, etc.) are reviewed in §4/§8; this section covers cross-cutting architecture.

HIGH Configuration Defaults

`get_config()` defaults to "development" when `FLASK_ENV` is unset, and `DevelopmentConfig` sets `DEBUG=True` and `JWT_COOKIE_SECURE=False`. The working-tree `backend/.env` currently has `FLASK_ENV=development`.

FILE `backend/config/settings.py:124-127,157-159` ; `backend/.env:1`

WHY If a deploy ever forgets to explicitly set `FLASK_ENV=production`, the app silently runs in dev mode: the refresh-token cookie loses its `Secure` flag (sendable over plain HTTP) and `DEBUG=True` risks the interactive Werkzeug debugger if anything invokes `app.run()`.

FIX Make production the default fallback, or fail startup loudly if `FLASK_ENV` is unset outside local dev. Assert `SECRET_KEY` / `JWT_SECRET_KEY` aren't placeholder values whenever `ENV=="production"`.

HIGH Deploy Configuration

`run.py` hardcodes `app.run(host="0.0.0.0", port=5000, debug=True)`, with nothing preventing production from being launched this way instead of via gunicorn.

FILE `backend/run.py:6`

WHY Gunicorn is in `requirements.txt`, implying it's the intended production entrypoint, but nothing enforces that. Launching with `python run.py` in production starts the single-threaded dev server bound to all interfaces with the interactive debugger enabled — a debugger reachable on any 500 is arbitrary code execution.

FIX Gate `debug` / `host` behind `app.config["DEBUG"]`, and document/enforce gunicorn as the only production start command (add a Procfile or a documented `gunicorn` invocation).

MEDIUM Rate Limiting / Infrastructure

`RATELIMIT_STORAGE_URI` defaults to `memory://`, and no `ProxyFix`/trusted-proxy configuration exists anywhere.

FILE `backend/config/settings.py:57` ; no `ProxyFix` in `backend/app/__init__.py`

WHY In-memory rate-limit storage is per-process — under gunicorn with `N` workers, the effective login/contact-form limit is `limit × N`. Separately, without `ProxyFix`, if the app sits behind any reverse proxy, both the rate-limit key and the IPs written to `audit_logs` (used for security investigation) reflect the proxy's address, not the real client's.

FIX Point `RATELIMIT_STORAGE_URI` at Redis in production; wrap the app in `werkzeug.middleware.proxy_fix.ProxyFix` configured with the correct trusted-hop count.

LOW Observability

No error-monitoring integration (Sentry or equivalent) anywhere — all error visibility is `logging.exception()` to `stdout`.

FILE `backend/app/middleware/logging_config.py` , `error_handlers.py:34-37`

WHY For a system serving real clients, `stdout`-only logs mean no alerting and no aggregation across restarts — a spike in 500s or repeated auth failures is invisible unless someone is actively tailing logs.

FIX Add Sentry (or similar) initialized in `create_app()` , at minimum for `ProductionConfig` .

✓ Strengths — backend architecture

- **Password hashing** uses Argon2 consistently across login, creation, reset, and self-service change — a strong, modern choice.
- **Refresh-token rotation with reuse detection**: every refresh issues a new token and revokes the old one; presenting an already-revoked token kills every active session for that admin. Textbook-correct.
- **Token storage discipline**: access token is Bearer-header-only, kept in JS memory (never `localStorage`); refresh token is `httpOnly`, `Secure` (in prod config), `SameSite=Strict`, path-scoped to `/api/auth/refresh` .
- Password reset/change **revokes all existing sessions** immediately.
- Login returns an identical error whether the email doesn't exist or the password is wrong — **no account-enumeration leak**.
- **Zero raw SQL** anywhere in the backend — exhaustively grepped, 100% ORM/query-builder usage.
- **Audit logging** writes in the same DB transaction as the mutation it records — no split-commit gap between an action and its trail.
- **CORS** is locked to an explicit origin allow-list (not `*`), correctly paired with `supports_credentials=True` .
- Centralized error handlers guarantee no stack trace or HTML error page ever reaches an API client.

SQLAlchemy models, 7 Alembic migrations, and the seed script. This is the strongest layer of the codebase.

MEDIUM Migration Drift

Three FK columns on `services` have indexes created by an explicit migration, but the model doesn't declare `index=True` on them — a drift the team already noticed and explicitly worked around once.

- FILE** `backend/app/models/service.py:34,53,69` vs `migrations/versions/e7ac351d16ab_...py:86-90` ; self-acknowledged in `migrations/versions/de4963600900_...py:80-84`
- WHY** Alembic autogenerate compares live DB state to model metadata. Since the model doesn't know these indexes exist, a future `alembic revision --autogenerate` will propose *dropping* them — a live, self-documented trap for whoever writes the next migration.
- FIX** Add `index=True` to those three columns in `service.py` to reconcile the model with reality. No new migration needed — the indexes already exist in the DB.

MEDIUM Indexing

`AuditLog` has separate single-column indexes on `admin_id` and `entity_type`, but no composite index on `(entity_type, entity_id)`.

- FILE** `backend/app/models/admin.py:29-30`
- WHY** The natural admin-panel query is "show the audit history for blog post #42" — `WHERE entity_type='blog_post' AND entity_id=42`. MySQL will use the single-column index and then scan every row of that type to filter by id — this gets slower every month as the insert-only `audit_logs` table grows.
- FIX** Add a composite index on `(entity_type, entity_id)` and drop the now-redundant single-column `entity_type` index.
- ```
op.create_index('ix_audit_logs_entity_type_id', 'audit_logs', ['entity_type', 'entity_id'])
```

#### MEDIUM Data Types / Timezones

Every timestamp column uses `db.DateTime(timezone=True)`, but MySQL's DATETIME type has no timezone storage — SQLAlchemy's MySQL dialect silently renders it as a plain, naive DATETIME.

- FILE** `backend/app/models/mixins.py:10-14` (pattern repeated in every model)
- WHY** The `timezone=True` flag gives a false sense of tz-safety. Any code comparing a naive value read from the DB against `datetime.now(timezone.utc)` raises `TypeError` or silently misinterprets the instant — this matters specifically for `Appointment.starts_at` / `reminder_24h_sent_at`, which a reminder-scheduling job is built around.
- FIX** Standardize on naive-UTC and re-attach `tzinfo=UTC` consistently at the ORM boundary, or move to a type that truly round-trips tz info. At minimum, audit the reminder job for naive/aware mixing.

#### MEDIUM Relationships / N+1

No eager-loading (joinedload/selectinload) exists anywhere in the codebase. The public blog list touches five separate lazy relationships per row, and the unpaginated list loads every published post.

FILE `backend/app/models/blog.py:74-82` (relationships default `lazy="select"`) consumed by `blueprints/blog/routes.py:8-35`

WHY With N published posts, `GET /blog` issues roughly `1 + 5N` queries. Fine at 20 posts, degrading linearly as the CMS accumulates years of content. `related_posts` additionally loads *every* published post into Python just to filter by category in memory.

FIX Use `selectinload` / `joinedload` on the list query; push the category filter into the SQL `WHERE` clause; add pagination to `/api/blog/posts`.

#### LOW Constraints

`Testimonial.rating` is protected by a CHECK constraint, which MySQL only enforces from 8.0.16 onward — silently parsed-but-ignored on older MySQL/MariaDB.

FILE `backend/app/models/testimonial.py:21-24`

FIX Confirm the production MySQL version is  $\geq 8.0.16$  as a go-live checklist item; add an application-level validator as a backstop if not guaranteed.

#### LOW Data Types

`FirmStat.value` is `String(40)` even though the suffix `(%, +)` is already split into its own column — a stray non-numeric string passes silently.

FILE `backend/app/models/firm.py:42-43`

FIX Change to `db.Integer` now, before real content accumulates — cheap today, expensive later.

#### LOW Maintainability

`TABLE_ARGS` (the InnoDB/utf8mb4 dict) is independently redefined in `auth.py` instead of imported from `admin.py` like every other model does.

FILE `backend/app/models/auth.py:17` vs `admin.py:4`

FIX `from app.models.admin import TABLE_ARGS` in `auth.py`; delete the local copy.

#### ✓ Strengths — database

- **Cascade design is correctly reasoned per relationship:** deleting a `BlogAuthor` orphans (not deletes) their posts via `SET NULL`; deleting a `JobOpening` preserves historical applications the same way; deleting an `Admin` keeps the audit trail ( `SET NULL` ) but revokes all sessions ( `CASCADE` ) — exactly the right call in each case.
- **Migration history is a single clean linear chain** across all 7 files — no branch points, no missing links. Unusually clean for a pre-launch project.
- **Backfill migrations are unusually careful:** nullable columns added first, backfilled per-row in Python, then locked to `NOT NULL` — the correct safe pattern for a populated table.
- `seed_data.py` **is genuinely idempotent** via natural-key `get_or_create`, and explicitly protects production data — it will not reset an existing admin's password.
- Sensible, deliberate use of `BigInteger` PKs for high-volume append-only tables vs `Integer` elsewhere; `JSON` reserved for genuinely schemaless payloads.
- **Webhook/Calendly idempotency** is well designed at the schema level — `WebhookEvent.dedupe_key` and `Appointment.calendly_event_id` are both unique+indexed (see §8 for why the webhook side is unused).

Database score: **8/10**

## API Testing & Domain Logic

Contact, career, blog, newsletter, services, team, testimonials, firm, and the admin CRUD blueprints. No live request/response testing was performed (see methodology note in §0) — findings below come from tracing each code path for its actual behavior under success, failure, and edge-case input.

### HIGH Scalability

POST `/api/admin/newsletter/send` loops synchronously over every subscriber and makes a blocking `send_email()` call per recipient inside the HTTP request — no queue or background worker exists anywhere in the repo.

FILE `backend/app/services/newsletter_service.py:140-189` ; `blueprints/admin/leads_routes.py:205-224`

WHY With a few hundred subscribers this exceeds typical gunicorn/proxy timeouts. If the worker is killed mid-loop, the campaign row is never committed even though some emails already went out — no record of a partial send, no way to resume without re-sending to everyone.

FIX Move the per-recipient loop to a background job (RQ/Celery/APScheduler); return 202 immediately with a campaign id the admin can poll for progress.

### MEDIUM Duplicate-Submission Handling

Unlike appointments (which upsert on `calendly_event_id`), Contact and Career submissions have no idempotency key at all.

FILE `backend/app/services/contact_service.py:10-36` ; `career_service.py:11-71`

WHY A double-click, a slow-network retry, or a resubmission within the rate-limit window creates two identical rows and fires two notification emails — the career flow even re-attaches the résumé each time.

FIX Add a short-lived dedupe check (same email+message hash within N minutes) or a client-generated idempotency key, mirroring the pattern already proven in `appointment_service.create_from_embed`.

### MEDIUM Response Consistency

Identical validation-failure bodies (`{"error": "Validation failed", "fields": {...}}`) are returned with a **different status code** depending on which blueprint produced them — 400 on public forms, 422 on admin CRUD.

FILE 400: `contact/controller.py:12` , `appointments/controller.py:22` , `career/controller.py:12` . 422: `newsletter/routes.py:19` , `admin/blog_routes.py:100` , `utils/admin_crud.py:70`

WHY A shared frontend error-handling utility keyed on status code ("422 → show field errors") will silently mishandle half of the API's forms.

FIX Pick one status code (422 is more semantically correct for "well-formed but invalid") and apply it uniformly, or document the split explicitly if it's intentional.

#### LOW Partial State on Failure

`save_resume()` / `save_media_image()` write the file to disk *before* the DB row commits, with no cleanup path if the commit then fails.

FILE `backend/app/services/career_service.py:20-47` ; `blueprints/admin/media_routes.py:82-98`

WHY A DB-level failure after a successful file write leaves an orphaned résumé/image on disk that no row references and the admin can never find or clean up.

FIX Wrap the DB write in try/except and delete the just-saved file on failure, or defer the physical save until after a successful `flush()` .

#### LOW Race Condition

Uniqueness checks (slugs, `FirmStat.key`) are select-then-insert with no `IntegrityError` fallback around the eventual commit.

FILE `backend/app/validations/content_validator.py:22-28` ; `utils/admin_crud.py:66-76`

WHY Two concurrent admin requests creating the same slug would have the second one hit a raw DB unique-constraint violation and return a 500 instead of the clean 422 the pre-check was meant to guarantee.

FIX Catch `IntegrityError` around the commit and translate it to the same 422 field-error shape as a fallback.

#### ✓ Strengths — API design

- `appointment_service.create_from_embed` is a genuinely well-built idempotent upsert (select-first, commit, `IntegrityError` fallback re-fetch on a race) — exactly the right pattern, and worth extending to `Contact/Career` (above).
- Newsletter subscribe correctly distinguishes new-subscribe (201) vs already-subscribed (200) vs reactivated (200) with honest status codes.
- `send_email()` is fully best-effort and never raises — a mail-provider outage can never roll back or 500 a form submission that already safely hit the database.
- MIME-sniffing (not just extension trust) on résumé and media uploads is real defense against renamed-file attacks.
- Consistent soft-delete → hard-delete two-step guard across awards, certifications, team members, testimonials, job openings, and newsletter subscribers, with a helpful 409 when deleting still-referenced media.
- `paginate_query` is a small, shared, defensively-coded utility used consistently across every admin list endpoint.

API / domain-logic score: **7/10**

# Security Audit

SQL injection, XSS, CSRF, auth/authz, secrets, uploads, rate limiting, CORS. The authentication core is strong (see §2 strengths); the gaps below are what stand between here and a clean bill of health.

## HIGH Broken Trust / Data Integrity

There is no Calendly webhook endpoint anywhere in the codebase. `POST /api/appointments/booked` — the only way appointments get created — accepts client-submitted name/email/phone plus a Calendly URI that is only checked for *looking like* a `*.calendly.com` link, never verified against Calendly's own records.

FILE `backend/app/blueprints/appointments/routes.py:13-18` ; `services/appointment_service.py:33-116` ; `validations/appointment_validator.py:30-34` . Confirmed by direct grep — zero webhook routes exist. `WebhookEvent` model and `CALENDLY_WEBHOOK_SIGNING_KEY` config exist but are never used.

WHY Anyone can POST a fabricated-but-well-formed Calendly URL with any name/email/phone, and the row is created with `status="confirmed"` — indistinguishable in the admin dashboard from a real booking. Full detail and fix in §8.

## MEDIUM File Upload / Stored XSS

Media Library uploads accept `.svg` with `image/svg+xml`, passing both the extension allow-list and the MIME-sniff check — confirmed directly in the source. Uploaded files are served back from the same origin as the API.

FILE `backend/app/utils/file_utils.py:24-31` ; served via `app/__init__.py:70-73`

WHY SVG can embed `<script>` . An "editor"-role account (lower trust than admin) could upload an SVG that executes JS if the file is ever opened directly or embedded via `<object>` / `<iframe>` — on the same origin where the non-httpOnly CSRF cookie lives.

FIX Drop `.svg` from the allow-list, or sanitize server-side (strip `<script>` /event handlers via `bleach` /an SVG sanitizer) before storing, and serve media with a strict CSP.

## MEDIUM Secrets / Git Hygiene

`frontend/.env` is tracked in git — confirmed via `git ls-files`. The root `.gitignore` only excludes `backend/.env`; the frontend has its own `.gitignore` that never mentions `.env` at all.

FILE `.gitignore:6` (backend-only rule); `frontend/.gitignore` (no env rule); tracked file: `frontend/.env`

WHY Today's contents are low-sensitivity (API base path, an internal LAN IP, a public Calendly URL) since Vite env vars are bundled into the client anyway — but the gitignore pattern itself is inconsistent and one careless future addition of a real key to this file will land directly in git history.

FIX Add `frontend/.env` to the root or frontend `.gitignore` , ``git rm --cached`` it, and rely on ``.env.example`` for onboarding — matching the pattern already correctly used on the backend.

#### LOW Anti-Abuse

RECAPTCHA\_SECRET is read into config but no code anywhere calls Google's siteverify endpoint or checks a token on any public form.

FILE `backend/config/settings.py:98` (dead config); no check in `contact/routes.py`, `newsletter/routes.py`, `career/routes.py`

WHY Public forms rely solely on per-IP rate limiting for anti-spam; a distributed botnet across many IPs can still flood the contact/career/newsletter endpoints and the outbound email provider.

FIX Wire up server-side recaptcha verification before accepting these submissions, or remove the unused config so it stops implying protection that isn't there.

#### MEDIUM Account Hardening

No 2FA/TOTP option exists for admin accounts — confirmed by grep, zero hits for TOTP/MFA anywhere in the backend.

WHY This is a professional-services admin panel handling real client PII (enquiries, job applications with résumés, appointment contact details); a single compromised admin password is the only thing standing between an attacker and that data, since password auth is the sole factor.

FIX Add optional TOTP-based 2FA for admin accounts, enforced for the "admin" role at minimum. See §10 for priority/rationale.

#### ✓ Strengths — security

- No SQL injection surface found anywhere — exhaustive grep confirms 100% parameterized ORM/query-builder usage.
- CORS is locked to an explicit origin allow-list, not `*`.
- Argon2 password hashing, refresh-token rotation with reuse/theft detection, memory-only access tokens, `httpOnly+Secure+SameSite=Strict` refresh cookie.
- Authorization coverage was independently re-verified for this report: every admin-only endpoint (Users, Security, Settings, Audit Log) is wrapped in `@require_role("admin")` server-side with no gaps — confirmed by direct inspection of all four route files.
- Résumés are never served from a guessable path — download requires the same admin JWT as everything else.
- Real `.env` secrets in the backend are gitignored and are high-entropy values, not placeholder defaults.

Security score: **6.5/10** — strong cryptographic and authorization foundations, held back by the unauthenticated appointment-creation path, config-default footgun, and the SVG upload gap.

## Performance Audit

This is the single weakest area of the project. Every issue here was independently verified — none are estimates.

### Homepage Hero

24.6MB PNG, eager, above the fold

### About / Blog Hero

9.88MB shared background PNG

### Public/ Folder Bloat

33MB unused image + 59MB placeholder video shipped to every build

### Code Splitting

None — every route in one bundle

### DB Queries

No eager-loading anywhere; unpaginated blog list

### Bulk Email

Synchronous, blocks a web worker per campaign

### Rate Limiter

In-memory, doesn't scale across gunicorn workers

### Caching

No HTTP caching headers or CDN in front of media uploads

See §1 and §3 for full findings and fixes on the frontend assets/bundling and the N+1 query patterns respectively — they're the substance of this score. Net effect: on a typical mobile connection, the homepage alone will fail Core Web Vitals badly enough to affect both user experience and Google ranking.

Performance score: **5/10**

## Admin Panel Review

CRUD, appointment/contact/newsletter management, auth, search, filters, pagination, and data sync — across all ~20 admin modules.

### MEDIUM Authorization (UX, not a data breach)

Admin-only pages (Users, Audit Log, Security, Settings) have no client-side role gate — a `RoleGuard` component exists but is never used anywhere in the route tree.

- FILE** `frontend/src/admin/routes/AdminRoutes.jsx:56-88` ; `auth/RoleGuard.jsx:8` (defined, zero usages — confirmed by grep); only `layouts/Sidebar.jsx:29-32` filters by role, and that's just hiding a nav link
- WHY** **Re-verified against the backend for this report:** every one of these endpoints is independently gated with `@require_role("admin")` server-side (confirmed in `users_routes.py` , `security_routes.py` , `settings_routes.py` , `audit_log_routes.py` ), so this is *not* a data-exposure hole — an "editor" navigating to `/admin/users` directly will have every API call rejected. The real problem is UX: they land on a broken error/empty-state page instead of a clean "you don't have access" redirect.
- FIX** Wrap the admin-only route subtree in `<RoleGuard allow=["admin"]>` so editors get redirected to the Dashboard with a clear message, instead of a confusing broken page. Keep the backend checks exactly as they are — they're correct.

### MEDIUM Correctness

The Security page's combined error-retry handler always re-fetches only the sessions list, never the failed-logins list.

- FILE** `frontend/src/admin/pages/Security/Security.jsx:22,41-43` — `refetchFailedLogins` is never destructured or used
- WHY** If only the failed-logins endpoint errors, clicking Retry re-fetches sessions (already fine) and leaves the page stuck on a full-page error state for failed logins until a hard reload.
- FIX** Destructure `refetch: refetchFailedLogins` and call both refetchers from the retry handler, or render two independent error states.

### MEDIUM CRUD Consistency

Delete-permission model is inconsistent across otherwise-similar modules: Team/Awards/Testimonials/JobOpenings require a deactivate step first and gate permanent delete to the "admin" role; BlogPosts and FirmStats expose a single-step, unconfirmed-by-role permanent delete to any authenticated user.

- FILE** `pages/BlogPosts/BlogPosts.jsx:104-111` , `pages/FirmStats/FirmStats.jsx:73`
- WHY** An editor can permanently delete a published, SEO-indexed blog post or a homepage stat with a single confirm click — the same class of user who can't even hard-delete an inactive team member in the sibling module.
- FIX** Apply the same two-step soft/hard-delete + admin-only gate used elsewhere to BlogPosts and FirmStats, or explicitly document why they're exempt.

**MEDIUM** Validation

The shared `TextField/TextAreaField/SelectField` components accept a `required` prop only to draw the red asterisk in the label — it's destructured and then never actually spread onto the underlying `<input>/<textarea>/<select>`.

FILE `frontend/src/admin/components/form/Field.jsx:25-36,38-50,52-60`

WHY Every "required" field across Team, Awards, Certifications, FirmStats, JobOpenings, BlogPosts, Services, Users, and Profile can be submitted empty — the only real enforcement is the server's 422 response after the fact, despite the UI visually implying otherwise.

FIX Spread `required` onto the underlying element in addition to using it for the label asterisk.

```
<input required={required} {...rest} />
```

**MEDIUM** Accessibility

`ConfirmDialog`, `Drawer`, and the newsletter recommendation modal set initial focus and handle Escape, but implement no focus trap.

FILE `components/ConfirmDialog.jsx:19-27 ; components/Drawer.jsx:7-18`

WHY A keyboard user confirming a delete can Tab past the dialog buttons onto a `DataTable` action or sidebar link hidden underneath the (only visually) blocking overlay, and activate it while the confirmation is still open.

FIX Add a small focus-trap cycling Tab/Shift+Tab between the first and last focusable element inside the open dialog — no such utility currently exists anywhere in the admin codebase.

**LOW** Accessibility

`TagMultiSelect`'s search input has a visual label with no `htmlFor/id/aria-label` link, unlike every field built via the shared form components.

FILE `components/TagMultiSelect.jsx:12,25-31`

FIX Add matching `id` / `htmlFor` or an `aria-label="Tags"`.

**LOW** Code Quality

`Dashboard.jsx` calls `useAsyncData(fetcher, [])`, but the hook's real signature is `useAsyncData(fetcher)` — the second argument is silently ignored.

FILE `pages/Dashboard/Dashboard.jsx:34` vs `hooks/useAsyncData.js:18`

WHY Not a live bug today, but it implies the hook supports a dependency array like `useEffect`, which it doesn't — a future page could rely on that and silently fail to refetch on a changed dependency.

FIX Remove the stray argument, or add real dependency-array support to the hook if that's the actual intent.

#### ✓ Strengths — admin panel

- `hooks/useAsyncData.js` is a clean, shared fetch/loading/error/refetch hook used identically across all ~20 list pages, correctly guarding against stale responses.
- `api/createResourceApi.js` is a single CRUD factory backing ~10 simple resources — strong DRY discipline, mirrored by a matching backend factory.
- Confirm-before-destroy is consistent almost everywhere — no one-click deletes found outside the two exceptions noted above.
- Loading states are real skeletons, not blank flashes; error states consistently offer a Retry button.
- Search, pagination, and status filters are present on every page with real data volume (Enquiries, Applications, Audit Log, Newsletter, Blog Posts, Appointments), with a properly debounced (300ms) server-side search.
- Every mutation refreshes its list via `refetch()` — no page anywhere requires a manual reload to see its own changes.
- The Login form has real `react-hook-form` validation, proper `aria-invalid` / `aria-describedby`, and distinguishes 429/401/network errors for the user.

Admin panel score: **7.5/10**

# Calendly Integration Review

Booking, storage, admin sync, duplicate handling, and failure modes.

**HIGH** Missing Webhook Receiver

The frontend embed is excellent (see §1 strengths), but the backend has no Calendly webhook endpoint at all. The *only* path to creating an Appointment row is a client-submitted POST that trusts whatever URI and contact details the browser sends.

**FILE** `backend/app/blueprints/appointments/routes.py:13-18` ; the `WebhookEvent` model ( `models/appointment.py:62-86` ) and `CALENDLY_WEBHOOK_SIGNING_KEY` config exist but neither is ever referenced by any route — confirmed by grep across the entire backend.

**WHY IT MATTERS** Booking authenticity is entirely dependent on trusting the browser. A visitor (or anyone who finds the endpoint) can submit a fabricated Calendly-shaped URL with any name/email/phone and it lands in the admin dashboard as a confirmed appointment, indistinguishable from a real one — no signature check exists because no webhook route exists to check one against.

**FIX** Implement the webhook receiver: verify the `Calendly-Webhook-Signature` HMAC against `CALENDLY_WEBHOOK_SIGNING_KEY` , dedupe via the `WebhookEvent.dedupe_key` the model already anticipates, and treat embed-submitted data as `pending` /unverified until the webhook confirms it — or explicitly flag embed-sourced appointments as unverified in the admin UI until this exists.

|                      |                                                                                                                                                                                 |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BOOKING (EMBED)      | Excellent — singleton script load, skeleton, retry-on-failure, popup-close tracking via MutationObserver.                                                                       |
| DB STORAGE           | Well-designed schema — <code>calendly_event_id</code> unique+indexed, denormalized display fields with a documented sync convention (see §3 finding on drift risk).             |
| DUPLICATE HANDLING   | Genuinely well-built idempotent upsert with an <code>IntegrityError</code> race fallback — the best-engineered write path in the API layer.                                     |
| ADMIN SYNC           | Works, but inherits the trust gap above — the admin dashboard can't distinguish a verified booking from a fabricated one.                                                       |
| API FAILURE HANDLING | Embed side handles script-load failure gracefully; the optional server-side Calendly API enrichment is disabled by default and read-only, so its failure doesn't block booking. |

Calendly integration score: **5/10** — the frontend and idempotency engineering are excellent; the missing webhook is the single fix that would move this to an 8+.

# Complete Functionality Testing

Honest scope note: this review traced every form, button, and flow through its actual code path, but did not click through a live, running instance end-to-end (that needs a provisioned MySQL database, a real Calendly account, and a working email provider — outside what this static review had access to). Concrete, already-confirmed functional gaps surfaced by the code trace are listed below; everything else needs the manual pass in the checklist.

## CONFIRMED FUNCTIONAL GAPS (FROM CODE TRACE)

- Unmatched URLs and unhandled render errors show React Router's generic crash screen, not a branded page (\$1).
- The Contact form's privacy-policy link goes to the homepage, not a policy document (\$1).
- Required admin form fields accept empty submission client-side; only the server catches it (\$7).
- The Security admin page can get permanently stuck on an error state for failed-login data specifically (\$7).
- Double-clicking Contact or Career submit (or a retried request) creates duplicate rows and duplicate emails (\$4).
- A large newsletter send can time out mid-broadcast with no record of how far it got (\$4).

## MANUAL QA CHECKLIST TO RUN BEFORE GO-LIVE

### BOOKING FLOW

Book a real Calendly slot end-to-end and confirm the admin dashboard reflects it correctly

### CONTACT FORM

Submit valid + invalid data, confirm email delivery and admin Enquiries list

### CAREER FLOW

Apply with a real résumé file, verify upload, download, and email notification

### NEWSLETTER

Subscribe, unsubscribe, resubscribe, and send a real test campaign to a small list

### ADMIN CRUD

Full create/edit/delete pass on every one of the ~20 modules with an editor *and* an admin account

### AUTH EDGE CASES

Expired session, concurrent tabs, password reset, revoke-session-then-retry

### MOBILE PASS

Every public page on an actual mid-tier Android device on a throttled connection

### SCREEN READER PASS

Tab through the Contact and Appointment forms and the admin ConfirmDialog with VoiceOver/NVDA

## Missing Features

Gaps that don't show up as "bugs" in the existing code because the code that would need to exist doesn't — evaluated against what a real production launch for a client-facing accounting firm needs.

### CRITICAL Calendly Webhook

Already detailed in §8 — repeated here because it's the single highest-priority missing piece, not just a bug in existing code.

**SOLVES** Removes the ability for anyone to inject fabricated appointments; makes the "Calendly Integration" the audit was asked to verify actually trustworthy.

### HIGH Automated Tests + CI

No test files exist for either the backend or frontend (confirmed — the only "test" hits are unrelated matches inside testimonials), and no CI configuration exists anywhere in the repo.

**WHY** Every fix in this report — including the Critical/High ones — currently has to be manually re-verified by hand on every future change, with no safety net against regressions.

**HOW** Start narrow: pytest coverage for the auth/refresh-rotation logic and the Calendly upsert (the two most failure-sensitive paths), a GitHub Actions workflow running lint + tests on every PR. Expand from there.

### HIGH Legal Pages

No Privacy Policy or Terms of Service page exists anywhere in the codebase, confirmed by direct search — yet the site collects names, phone numbers, emails, and résumés from real people.

**WHY** A firm that collects personal data without a real, linked privacy policy is exposed under India's DPDP Act 2023 (and general user trust) — this is a business/legal risk, not a code-quality nitpick.

**HOW** Have the firm's counsel draft the actual policy text; build `/privacy-policy` and `/terms` routes; link them from the Contact form, footer, and Career application.

### MEDIUM Error Monitoring

No Sentry (or equivalent) integration — see §2.

**HOW** Add the Sentry SDK to `create_app()` for production, and a lightweight frontend equivalent (Sentry's React SDK or even just a global `window.onerror` reporter) for the public site and admin panel.

### MEDIUM Admin 2FA

See §5 — no second factor exists for admin login.

**HOW** TOTP-based (Google Authenticator-compatible) second factor, enforced for the "admin" role; "editor" can remain optional initially.

**MEDIUM** SEO Infrastructure

No robots.txt or sitemap.xml anywhere in frontend/public/, and no analytics tag (Google Analytics/Search Console or similar) anywhere in index.html — both confirmed by direct inspection.

**WHY** Without a sitemap, search engines discover the site's ~30 routes slowly and unreliably; without analytics, there's no visibility into which pages/forms real visitors actually use once live.

**HOW** Generate a sitemap at build time (or a static one, given the route list is small and mostly static), add a basic robots.txt, and wire up GA4 or a privacy-respecting alternative.

**LOW** Admin Session UX

The proactive-refresh scheduler silently logs a session out on failure with no advance warning to the admin mid-task.

**HOW** A "your session is about to expire" toast a minute before forced logout gives an admin mid-form-edit a chance to save.

## 11 Production Readiness

Is this safe to point real client traffic at today?

**Not yet — but it's close, and the path is short.** The parts of this system that are hardest to get right in a rewrite — password/token handling, refresh rotation, database cascade design, migration discipline, CRUD scaffolding — are already done well. What's missing is mostly the last-mile production checklist: pin the config defaults, compress three images, add the webhook, and put a background job behind the newsletter send. None of that requires re-architecting anything.

Production readiness score: **6/10**



# Final Action Plan

Everything above, deduplicated and ordered by priority. Fix top-down.

▼ CRITICAL — BLOCK LAUNCH ON THESE

3 items

|                                                                             |                                                                                                                                            |                                                    |
|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| <b>Compress/replace the 24.6MB homepage hero image</b>                      | and the other oversized public assets (9.9MB About/Blog background, 59MB placeholder video, 33MB unused image).                            | HomeHero.jsx, AboutHero.jsx, BlogHero.jsx, public/ |
| <b>Fix the placeholder document title and missing per-page SEO metadata</b> | across ~12 routes.                                                                                                                         | index.html, ~12 page components                    |
| <b>Pin the config-default footgun</b>                                       | : DevelopmentConfig , run.py 's debug=True / host="0.0.0.0" must be reachable in prod.<br>production and<br>must<br>never silently inherit | config/settings.py, run.py                         |

▼ HIGH — FIX BEFORE REAL TRAFFIC

8 items

|                                                          |                                                                                     |                                         |
|----------------------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------|
| <b>Implement the Calendly webhook receiver</b>           | with signature verification, and stop trusting client-submitted booking data as-is. | appointments/routes.py                  |
| <b>Add a branded 404 page and router errorElement</b>    | .                                                                                   | AppRoutes.jsx                           |
| <b>Move newsletter sending to a background job</b>       | , return 202 + pollable status.                                                     | newsletter_service.py                   |
| <b>Add route-level code splitting</b>                    | ( React.lazy ) across the public site's ~20 routes.                                 | AppRoutes.jsx                           |
| <b>Fix N+1 queries</b>                                   | on the blog/services/team/testimonials list endpoints; paginate the blog list.      | blog/routes.py, admin/service_routes.py |
| <b>Stand up a minimal test suite + CI pipeline</b>       | , starting with auth and the Calendly upsert path.                                  |                                         |
| <b>Build real Privacy Policy and Terms pages</b>         | and link them from the Contact form.                                                | ContactForm.jsx:339                     |
| <b>Add idempotency to Contact and Career submissions</b> | to stop duplicate rows/emails on retry.                                             | contact_service.py, career_service.py   |

▼ MEDIUM — FIX WITHIN THE FIRST POST-LAUNCH SPRINT

14 items

|                                                           |                                        |               |
|-----------------------------------------------------------|----------------------------------------|---------------|
| <b>Move rate-limit storage to Redis and add ProxyFix</b>  | for correct client IPs behind a proxy. | settings.py   |
| <b>Drop or sanitize SVG uploads in the Media Library.</b> |                                        | file_utils.py |

Untrack `frontend/.env` from git and fix the gitignore pattern. `.gitignore`

Add `<RoleGuard>` to admin-only routes for a clean redirect instead of a broken page for editors. `AdminRoutes.jsx`

Fix the Security page's broken failed-logins retry. `Security.jsx`

Make BlogPosts/FirmStats delete permissions consistent with the rest of the admin panel. `BlogPosts.jsx`, `FirmStats.jsx`

Actually apply the `required` prop to admin form inputs, not just the label asterisk. `form/Field.jsx`

Add a focus trap to `ConfirmDialog` / `Drawer` . `ConfirmDialog.jsx`

Reconcile the `services` model with its already-existing indexes to stop migration drift. `models/service.py`

Add a composite index on `AuditLog(entity_type, entity_id)` . `models/admin.py`

Audit and fix naive/aware datetime handling around the appointment-reminder job. `mixins.py`, `appointment_service.py`

Add Sentry (or equivalent) error monitoring. `app/__init__.py`

Add TOTP-based 2FA for admin accounts.

Add `robots.txt` / `sitemap.xml` and an analytics tag.

▼ LOW – POLISH, TACKLE OPPORTUNISTICALLY

9 items

Standardize decorative image `alt` / `aria-hidden` usage sitewide.

Extract shared form/animation boilerplate out of `ContactForm/BookingSection/ApplyNow`.

Delete the unused `GenericServicePage.jsx` .

Import `TABLE_ARGS` in `auth.py` instead of redefining it.

Change `FirmStat.value` to an integer type.

Add an `IntegrityError` fallback around admin slug/key uniqueness checks.

Delete just-saved files on a DB commit failure in `résumé/media` uploads.

Unify the 400 vs 422 validation status code split across public vs admin blueprints.

Give `TagMultiSelect` 's search input a real label association.

Reviewed 1 Aug 2026 · Findings independently spot-verified where marked · This report reflects a static code audit, not a live penetration test or load test.